TASK 21– PHP WEB APPLICATION WITH APPLICATION LOAD BALANCER AND AUTO SCALING

Objective

1. Host a simple PHP web application in an AWS EC2 instance.
2. Configure an Application Load Balancer (ALB) for the instances running the web application.
3. Create an Auto Scaling Group (ASG) equipped with dynamic scaling policies for the application.
4. Attach the Load Balancer to the Auto Scaling Group's Target Group.
5. Verify the scaling policies by manually increasing and decreasing the workload.

1. Create a Security Group

A Security Group was created to control inbound traffic and allow HTTP access to the PHP web application.

- **Steps**:
    1. Navigate to the Amazon EC2 Console and select **Network & Security** > **Security Groups**.
    2. Click **Create security group**.
    3. Configure the parameters as follows:
        - **Security Group Name**: web-app-sg
        - **Description**: Allow HTTP web traffic
        - **VPC**: Default VPC
    4. Under **Inbound rules**, add an HTTP rule allowing port 80 from 0.0.0.0/0.
    5. Leave outbound rules as default and click **Create security group**.

2. Create a Launch Template

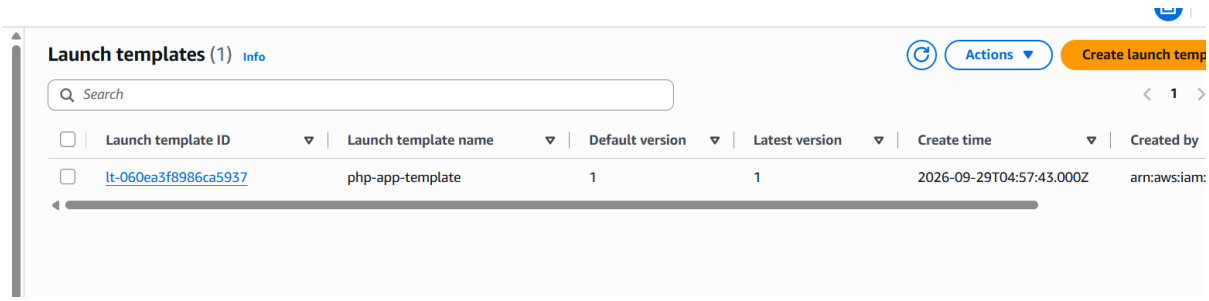
A Launch Template was created to standardize instance deployments, ensuring that any dynamically scaled instances inherit the correct operating system, instance type, security group, and initialization script.

- **Steps**:
    1. From the EC2 Dashboard, select **Launch Templates** > **Create launch template**.
    2. Enter the template name: php-web-app-template.
    3. Choose **Amazon Linux 2023** as the AMI and select instance type t2.micro.
    4. Select **Proceed without a key pair** and assign the security group web-app-sg.
    5. Expand **Advanced details**, navigate to **User data**, and input the setup script:

Bash

```
#!/bin/bash
# Update the operating system
dnf update -y
# Install Apache web server and PHP
dnf install -y httpd php
# Start Apache and make it run on system boot
systemctl start httpd
systemctl enable httpd
# Create the PHP script that displays the local IP address
cat <<EOF > /var/www/html/index.php
<!DOCTYPE html>
<html>
<head>
<title>PHP Web Application</title>
</head>
<body>
<h1>Hello World!</h1>
<p><strong>Server IP Address:</strong> <?php echo
\$_SERVER['SERVER_ADDR']; ?></p>
</body>
</html>
EOF
```

6. Click **Create launch template**.



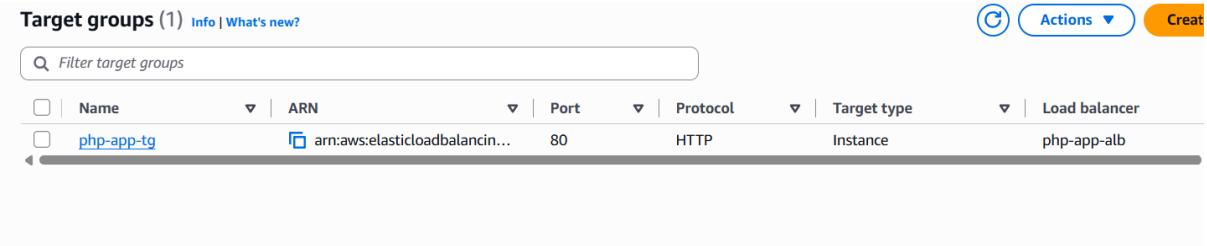
3. Create the Application Load Balancer and Target Group

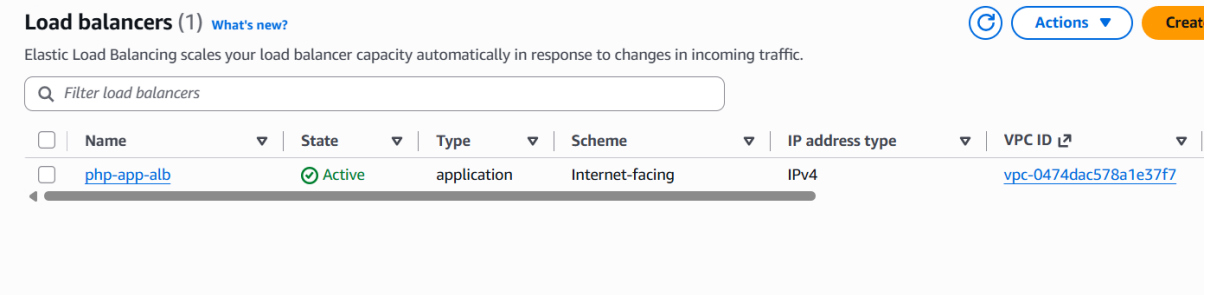
An Application Load Balancer (ALB) was provisioned to distribute incoming user traffic across the underlying backend web servers.

- **Steps for Target Group**:
    1. Navigate to **Target Groups** > **Create target group**.
    2. Choose **Instances** as the target type, name it php-web-app-tg, select protocol HTTP on port 80, and choose the default VPC.
    3. Skip registering targets manually and click **Create target group**.

Steps for Load Balancer:

1. Navigate to **Load Balancers** > **Create load balancer** > select **Application Load Balancer**.
2. Configure the basic settings:
    - **Name**: php-web-app-alb
    - **Scheme**: Internet-facing
    - **VPC & Subnets**: Default VPC with at least two availability zones selected.
3. Select the web-app-sg security group.
4. Under **Listeners and routing**, configure HTTP port 80 to forward traffic to php-web-app-tg.
5. Click **Create load balancer**.

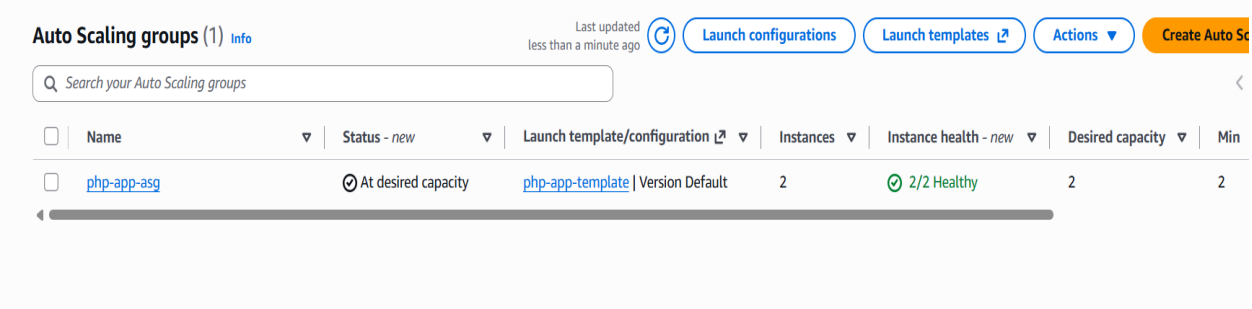




4. Create the Auto Scaling Group

An Auto Scaling Group (ASG) was established to manage dynamic instance scaling, relying on health checks and target tracking policies.

- **Steps**:
    1. Navigate to **Auto Scaling Groups** > **Create Auto Scaling group**.
    2. Enter ASG name php-web-app-asg and select the php-web-app-template launch template.
    3. Choose the default VPC and select matching Availability Zones/subnets.
    4. Under **Load balancing**, select **Attach to an existing load balancer** and choose php-web-app-tg.
    5. Enable **Elastic Load Balancing health checks**.
    6. Configure capacity parameters:
        - **Desired capacity**: 2
        - **Minimum capacity**: 2
        - **Maximum capacity**: 3
    7. Under **Automatic scaling**, choose **Target tracking scaling policy**:
        - **Policy name**: Target Tracking Policy
        - **Metric type**: Average CPU utilization
        - **Target value**: 15 (15%)
        - **Instance warmup**: 60 seconds
    8. Review the configurations and click **Create Auto Scaling group**

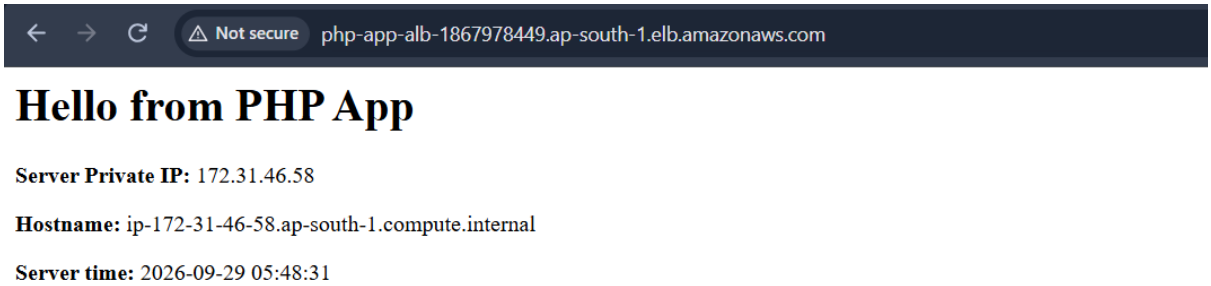


5. Verify the PHP Application Through the Load Balancer

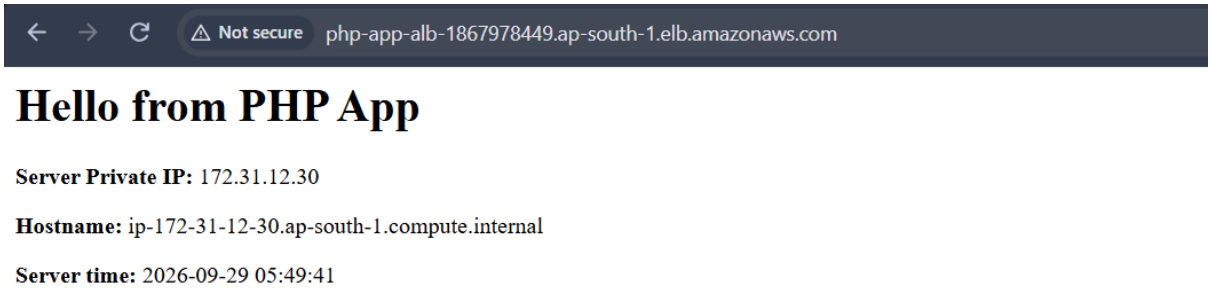
The deployed application was accessed via the ALB DNS endpoint to confirm traffic distribution.

- **Steps**:
    1. Copy the DNS name of php-web-app-alb from the EC2 console.
    2. Open the URL in a web browser: http://php-web-app-alb-<dns-suffix>.amazonaws.com.
    3. The browser successfully displayed Hello World! along with the dynamic private IP address of the responding instance. Refreshing the page showed alternating IP addresses, proving that requests were successfully load-balanced across the instances.

First server:



Second server:



6. Manually Increase the Load (Scale-Out Test)

To verify the auto-scaling capability, artificial CPU load was introduced to trigger the 15% threshold policy.

- **Steps**:
    1. Connect to one of the running EC2 instances via **EC2 Instance Connect**.
    2. Install the stress utility package:

       Bash

       sudo dnf install -y stress

    3. Run the stress command to generate CPU utilization for 10 minutes:

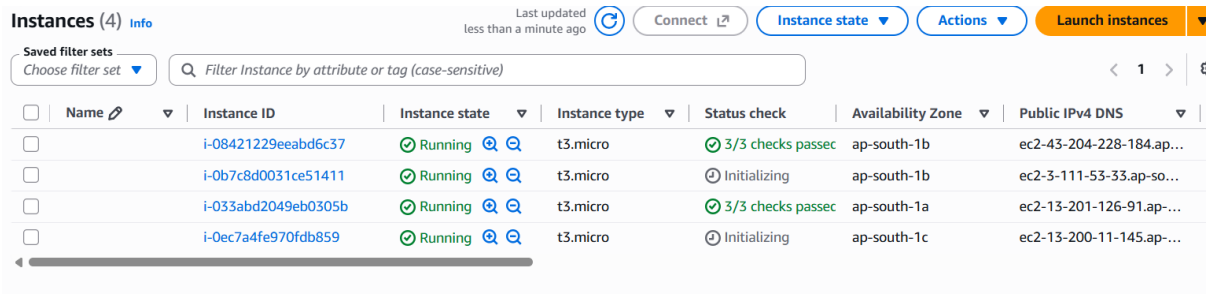
       Bash

       stress --cpu 1 --timeout 600

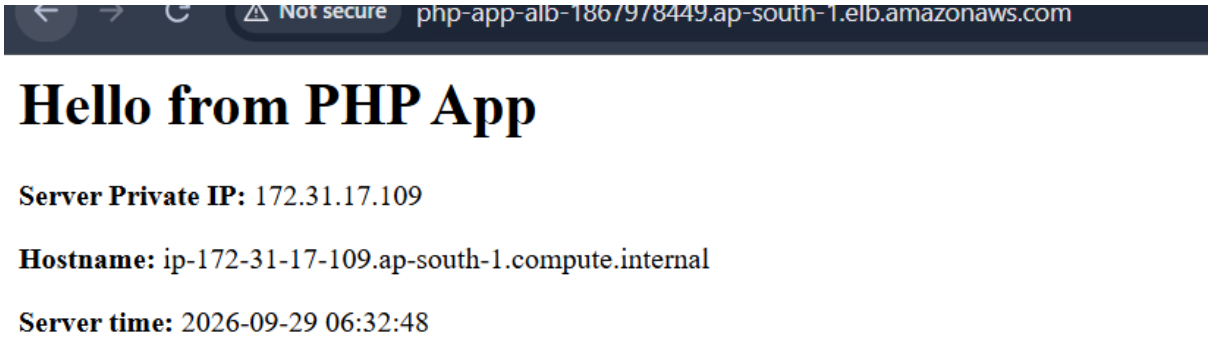
4. Navigate to the **Auto Scaling groups** console, select php-web-app-asg, and check the **Activity** tab.
5. When the average CPU utilization exceeded the target value of 15%, the ASG automatically initiated a scale-out operation, increasing the instance count from **2 instances to 3 instances** by launching a new EC2 instance via the launch template.

```
Last login: Tue Sep 29 05:57:02 2026 from 49.47.198.161
[ec2-user@ip-172-31-12-30 ~]$ stress --cpu 2 --timeout 300
stress: info: [28960] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [28960] successful run completed in 300s
```

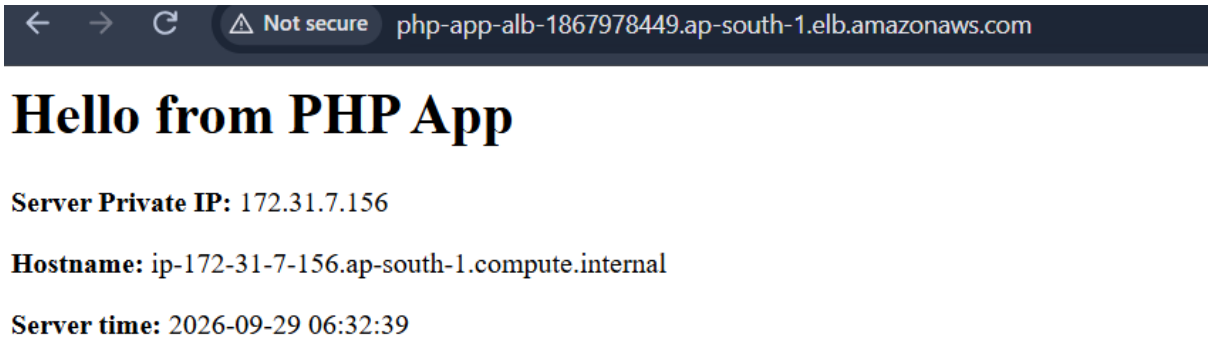
Instance launched:



New server:



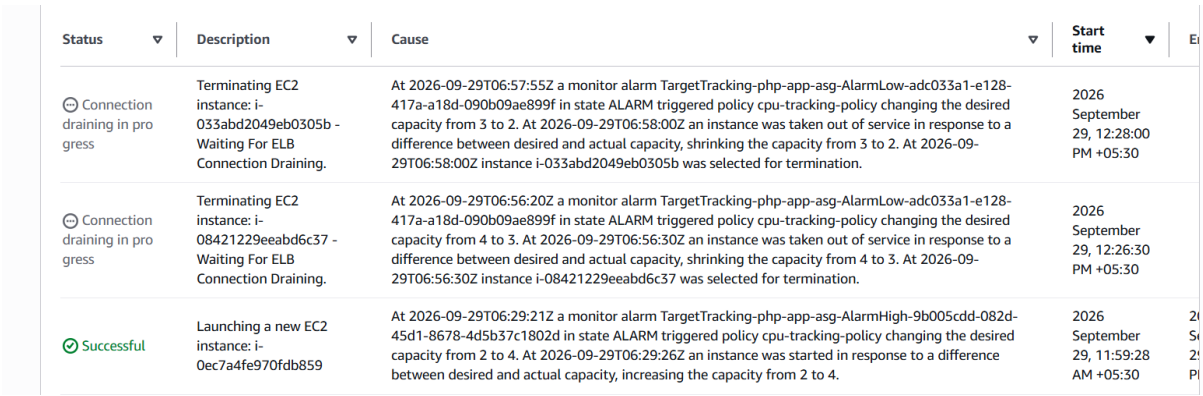
New server:



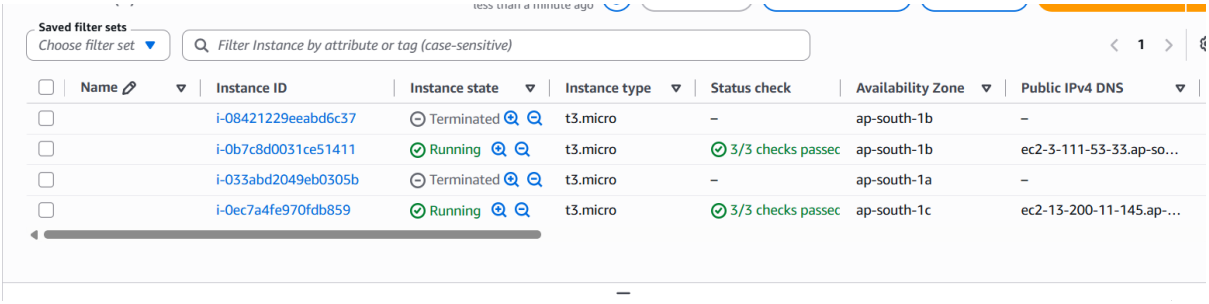
7. Verify Scale-In Behavior

After stopping the stress test, the infrastructure automatically scaled back down to its original state.

- **Steps**:
    1. Terminate the running stress process in the terminal using Ctrl + C.
    2. As CPU utilization dropped back down, the ASG continuously monitored the metrics.
    3. Once conditions allowed capacity reduction, a scale-in activity triggered, gracefully terminating the excess instance and returning the group size from **4 instances back to 2 instances**.



Status	Description	Cause	Start time
Connection draining in progress	Terminating EC2 instance: i-033abd2049eb0305b - Waiting For ELB Connection Draining.	At 2026-09-29T06:57:55Z a monitor alarm TargetTracking-php-app-asg-AlarmLow-adc033a1-e128-417a-a18d-090b09ae899f in state ALARM triggered policy cpu-tracking-policy changing the desired capacity from 3 to 2. At 2026-09-29T06:58:00Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 3 to 2. At 2026-09-29T06:58:00Z instance i-033abd2049eb0305b was selected for termination.	2026 September 29, 12:28:00 PM +05:30
Connection draining in progress	Terminating EC2 instance: i-08421229eeabd6c37 - Waiting For ELB Connection Draining.	At 2026-09-29T06:56:20Z a monitor alarm TargetTracking-php-app-asg-AlarmLow-adc033a1-e128-417a-a18d-090b09ae899f in state ALARM triggered policy cpu-tracking-policy changing the desired capacity from 4 to 3. At 2026-09-29T06:56:30Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 4 to 3. At 2026-09-29T06:56:30Z instance i-08421229eeabd6c37 was selected for termination.	2026 September 29, 12:26:30 PM +05:30
Successful	Launching a new EC2 instance: i-0ec7a4fe970fdb859	At 2026-09-29T06:29:21Z a monitor alarm TargetTracking-php-app-asg-AlarmHigh-9b005cdd-082d-45d1-8678-4d5b37c1802d in state ALARM triggered policy cpu-tracking-policy changing the desired capacity from 2 to 4. At 2026-09-29T06:29:26Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 2 to 4.	2026 September 29, 11:59:28 AM +05:30



Conclusion

This task successfully demonstrated the complete lifecycle of cloud deployment: hosting a PHP web application on Amazon EC2, distributing incoming traffic using an Application Load Balancer, enforcing capacity limits with an Auto Scaling Group, and validating automated scaling responses through stress testing.